Komut sistemi kılavuzu

LED’i terminalden yönetmek

PuTTY’de yazılan bir komut; UART, DMA, yazılım halkası ve parser’dan geçip yeşil LD3 LED’ine nasıl ulaşıyor, cevabı ekrana nasıl dönüyor?



115200
baud, 8N1

9
terminal komutu

256 + 2048
bayt alım tamponu

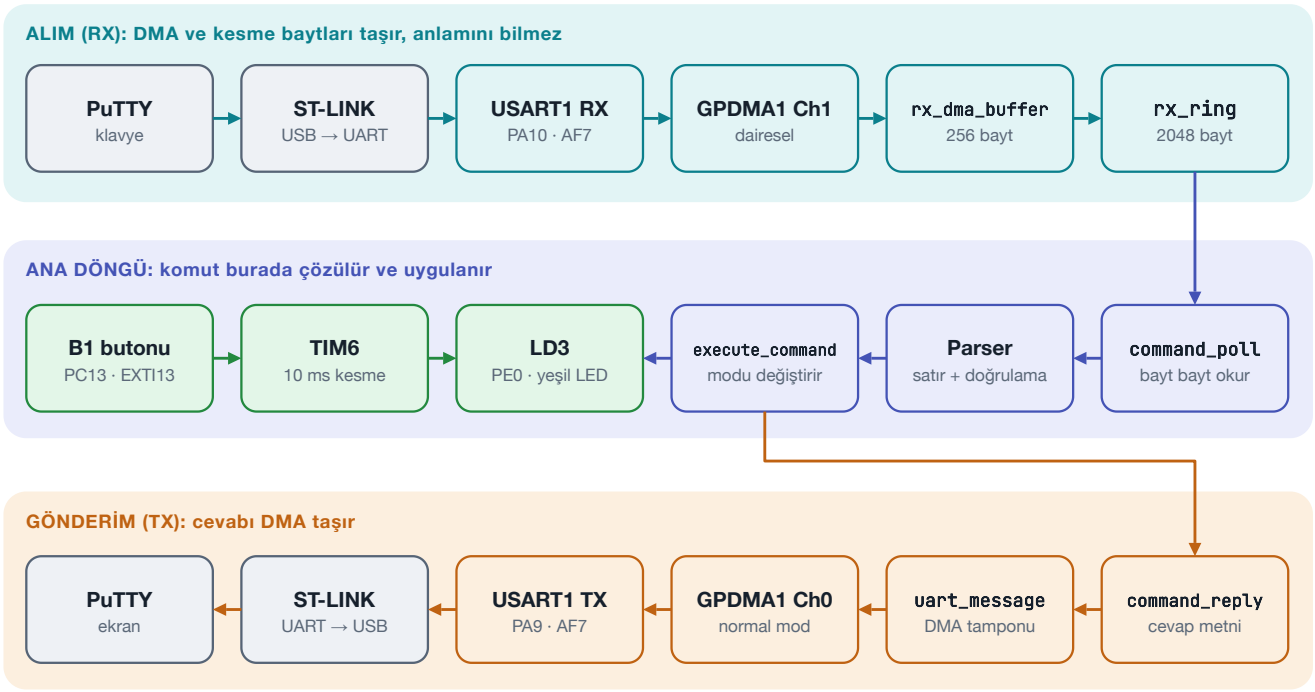
69 + 88
otomatik kontrol

3 Kasım 2026

1 · BÜYÜK RESİM

Bir komutun tam yolu

Firmware, PuTTY'den gelen metni **DMA ile alır**, **ana döngüde çözer**, LED'i değiştirir ve cevabı yine **DMA ile gönderir**. CPU, karakter başına kesmeyle uğraşmaz; ekran çizimi de bu sırada çalışmaya devam eder.



Renkler kılavuz boyunca aynı anlamdadır: **turkuaz** alım, **lacivert** ana döngü, **turuncu** gönderim, **yeşil** LED ve kontrol.

Tasarımı belirleyen dört karar

DMA anlam bilmez

DMA yalnızca baytları USART1 ile RAM arasında taşır. Komutun ne olduğuna parser karar verir.

IDLE komut sonu değildir

Hat boşalınca gelen bildirim yalnızca "yeni bayt var" demektir. Komutu CR veya LF tamamlar.

Kesme kısa, ana döngü işçi

Kesmeler baytı kuyruğa koyar. Komut çözme ve `snprintf` ana döngüde kalır.

Parser donanımdan bağımsız

`watch_led_commands.c` HAL, DMA veya GPIO çağırmaz, bu yüzden bilgisayarda da test edilir.

2 · KULLANIM

PuTTY ile bağlanmak

1. **Firmware'i yükle.** `Firmware/Smartwatch.dfw` paketini Device Studio ile karta yükle. Uygulama tabanı `0x0800A000`'dır.
2. **Kartı bağla.** ST-LINK USB **CN5** portunu kullan. Port Mac'te `/dev/cu.usbmodem…` olarak görünür. Device Studio aynı portu kullandığı için kapalı olmalı.
3. **PuTTY'yi aç.** Proje kökünden aşağıdaki betiği çalıştır: portu bulur, `STM32-LED` oturumunu aşağıdaki ayarlarla yazar ve PuTTY'yi açar.

```
tools/led-terminal.sh
```

Bağlantı ayarları

Hız	115200
Veri / Parite / Stop	8 / None / 1
Akış kontrolü	None
Local echo	Force on
Local line editing	Force on

Firmware yazılanı geri yansıtmaz ve ham Backspace karakterini reddeder. Son iki ayar bu yüzden PuTTY tarafında açıktır: satırı PuTTY düzenler, Enter'da gönderir.

Komutlar

`LED ON`	Sürekli yakar
`LED OFF`	Sürekli söndürür
`LED TOGGLE`	Tersine çevirip sabit tutar
`LED BLINK`	Saklı aralıkla yanıp söner
`SET INTERVAL 250`	Aralığı ayarlar, yanıp söner
`LED STATUS`	`GET STATUS` ile aynı
`GET STATUS`	Durum satırı verir
`RESET PRESSES`	Buton sayacını sıfırlar
`HELP`	Komut listesini verir

Komutlar büyük harflidir. Aralık 10–60000 ms olmalı ve 10'un katı olmalıdır. Ayarlar RAM'de tutulur; yeniden başlatınca 1000 ms yanıp sönmeye dönülür.

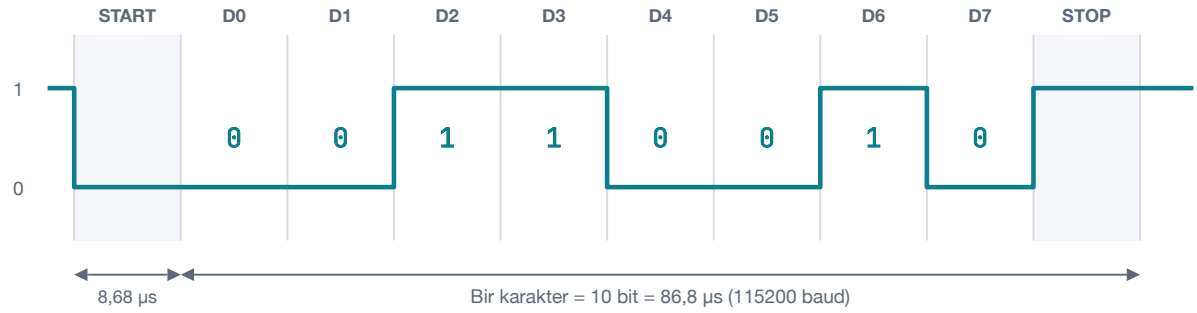
Örnek oturum · sayaçlar çalışma anına göre değişir

```
> HELP
OK HELP: LED ON|OFF|TOGGLE|BLINK|STATUS; SET INTERVAL <10..60000 ms, step 10>; GET STATUS; RESET PRESSES;
HELP
> LED TOGGLE
OK LED=LD3 mode=OFF state=OFF toggle_ms=0 cycle_ms=0 presses=0 uptime_ms=4170 uart_ok=4 uart_errors=0
interval_ms=250 rx_errors=0 rx_overflows=0
LED=LD3 mode=OFF state=OFF toggle_ms=0 cycle_ms=0 presses=0 uptime_ms=5000 uart_ok=5 ... ← her saniye gelen
durum satırı
> SET INTERVAL 15
ERR interval_range 10..60000 step 10
```

3 · FİZİKSEL KATMAN

Bitten bayta: USART1

Mac'teki PuTTY veriyi USB'den gönderir. Karttaki ST-LINK bunu UART sinyaline çevirir ve **PA10** pininden USART1'e verir. Cevap ters yönde **PA9** üzerinden döner. Her karakter hatta 10 bit olarak gider: başlangıç biti, en düşük anlamlı bitten başlayarak 8 veri biti ve bitiş biti.



"L" harfi (`0x4C`). UART yalnızca karakterin sınırını belirler; komutun nerede bittiğini üstteki metin protokolü söyler.

RX DMA kurulumu

STM32U5'in GPDMA birimi dairesel çalışmayı **linked-list** ile yapar. Kod, kendisine bağlı tek bir düğüm kurar. Böylece tampon dolunca DMA yeniden başlatılmadan başa döner ve alım hiç kesilmez.

Core/Src/led_button_demo.c satır 105–117 — *TX ayarı temel alınır, yön ve artırma ters çevrilir*

```
105     DMA_NodeConfTypeDef node = {0};
106     node.NodeType = DMA_GPDMA_LINEAR_NODE;
107     node.Init = led_demo_uart_tx_dma.Init;
108     node.Init.Request = GPDMA1_REQUEST_USART1_RX;
109     node.Init.Direction = DMA_PERIPH_TO_MEMORY;
110     node.Init.SrcInc = DMA_SINC_FIXED;
111     node.Init.DestInc = DMA_DINC_INCREMENTED;
112     node.TriggerConfig.TriggerPolarity = DMA_TRIG_POLARITY_MASKED;
113     node.DataHandlingConfig.DataExchange = DMA_EXCHANGE_NONE;
114     node.DataHandlingConfig.DataAlignment = DMA_DATA_RIGHTALIGN_ZEROPADDED;
115     if(HAL_DMAEx_List_BuildNode(&node, &rx_node) != HAL_OK ||
116        HAL_DMAEx_List_InsertNode_Tail(&rx_list, &rx_node) != HAL_OK ||
117        HAL_DMAEx_List_SetCircularMode(&rx_list) != HAL_OK) Error_Handler();
```

Core/Src/led_button_demo.c satır 62–66

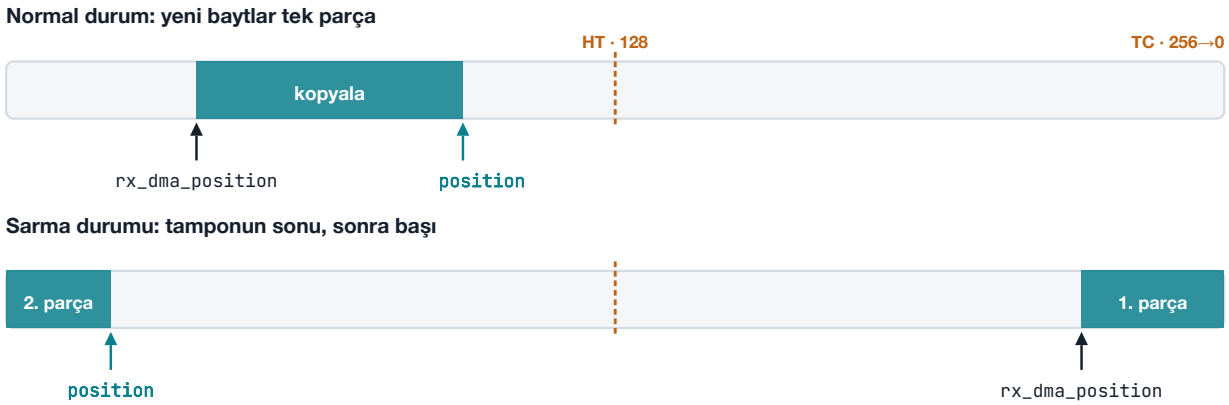
```
62 static HAL_StatusTypeDef uart_receive_start(void)
63 {
64     rx_dma_position = 0;
65     return HAL_UARTEx_ReceiveToIdle_DMA(&led_demo_uart, rx_dma_buffer, sizeof rx_dma_buffer);
66 }
```

> **Neden 256 bayt yetiyor?** Bir karakter 86,8 µs sürdüğü için 128 baytlık yarım tampon yaklaşık 11 ms'de dolar. HT ve TC bildirimleri tamponu bu süre içinde boşaltır; kesmeler bundan uzun süre engellenmemelidir.

4 · ALIM

DMA bildirimleri: HT, TC ve IDLE

DMA arka planda tampona yazarken üç olay aynı callback'i çağırır: **HT** (yarım tampon), **TC** (tam tampon) ve USART'ın **IDLE** olayı (hat boşaldı). Kısa bir komutta tampon dolmaz, bu yüzden veriyi çoğunlukla IDLE teslim eder.



Callback son işlenen konum ile DMA'nın anlık konumu arasındaki baytları kopyalar. Sarma varsa önce sonu, sonra başı kopyalar.

Core/Src/led_button_demo.c satır 280–296

```
280 void HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *uart, uint16_t size)
281 {
282     if(uart != &led_demo_uart || !size || size > RX_DMA_SIZE) return;
283     /* Read the live DMA position: a pending HT/TC interrupt can be serviced
284      * after IDLE has already delivered newer bytes. Size alone would replay
285      * stale data. IDLE is a delivery hint, never a command boundary. */
286     uint16_t position = RX_DMA_SIZE - __HAL_DMA_GET_COUNTER(&led_demo_uart_rx_dma);
287     if(position == RX_DMA_SIZE) position = 0;
288     if(position == rx_dma_position) return;
289     __DMB();
290     if(position < rx_dma_position) {
291         receive_bytes(rx_dma_buffer + rx_dma_position, RX_DMA_SIZE - rx_dma_position);
292         rx_dma_position = 0;
293     }
294     receive_bytes(rx_dma_buffer + rx_dma_position, position - rx_dma_position);
295     rx_dma_position = position;
296 }
```

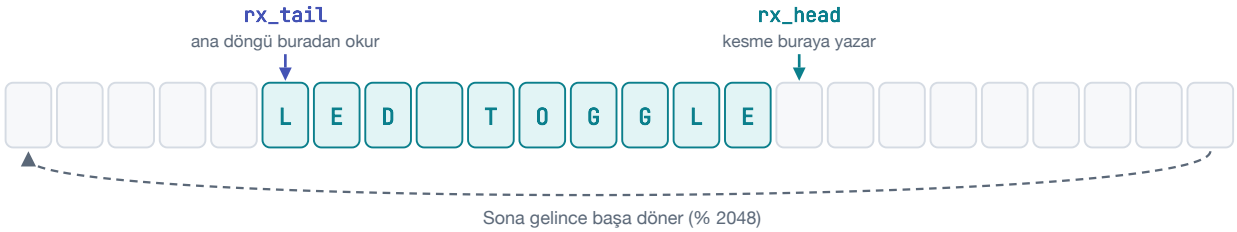
Neden anlık konum okunuyor?

Callback'e gelen `size` değerine güvenilmez. Bekleyen bir HT veya TC kesmesi, IDLE daha yeni baytları teslim ettikten sonra çalışabilir; o değer kullanılsa eski baytlar ikinci kez işlenirdi. Kod bunun yerine DMA'nın kalan sayacını okur. Konum değişmediyse hiçbir şey yapmaz, yani aynı olay iki kez gelse bile bayt tekrar etmez.

5 · KUYRUK

Yazılım halkası: kesmeden ana döngüye

Ana döngü LVGL ile ekranı da çizdiği için bir süre meşgul kalabilir, DMA ise beklemez. Araya konan **2048 baytlık halka** bu farkı karşılar: kesme yazar, ana döngü okur.



Tek üretici, tek tüketici. Bir yuva boş bırakılır, bu yüzden kapasite 2047 bayttır.

Core/Src/led_button_demo.c satır 268–278

```
268 static void receive_bytes(const uint8_t *data, uint16_t size)
269 {
270     if(rx_overflow_pending || rx_error_pending) return;
271     for(uint16_t i = 0; i < size; ++i) {
272         uint32_t next = (rx_head + 1U) % RX_RING_SIZE;
273         if(next == rx_tail) { rx_overflow_pending = 1; return; }
274         rx_ring[rx_head] = data[i];
275         __DMB();
276         rx_head = next;
277     }
278 }
```

`__DMB()` bellek bariyeridir: `rx_head` ilerlemeden önce bayt RAM'e yazılmış olur, yani ana döngü eksik bayt okuyamaz. Halka dolarsa yeni bayt yazılmaz, taşma bayrağı kalkar ve ana döngü bunu `ERR rx_data_lost` olarak bildirir (10. bölüm).

Ana döngü: komutu çeken taraf

Komut işleme ile ekran çizimi aynı döngüyü paylaşır. `led_button_demo_poll()` her turda önce hataları toparlar, sonra `command_poll()` ile halkadan okur, en sonda bekleyen cevabı gönderir.

Core/Src/main.c satır 33–38

```
33      while(1) {
34          led_button_demo_poll();
35          device_runtime_poll();
36          if(HAL_GetTick()-last_update>=1000) { refresh_rtc(); last_update=HAL_GetTick();
    ++smartwatch_heartbeat; }
37          watch_platform_display_poll();
38          lv_timer_handler();
```

Tüketici: command_poll

Her çağrıda en fazla 256 bayt ve **en fazla bir tamamlanmış komut** işlenir. Fonksiyon `if(reply_length) return;` ile başlar: bekleyen bir cevap varken yeni komut çalıştırılmaz, cevabı yazacak yer açılana kadar komut halkada bekler.

Core/Src/led_button_demo.c satır 355–382

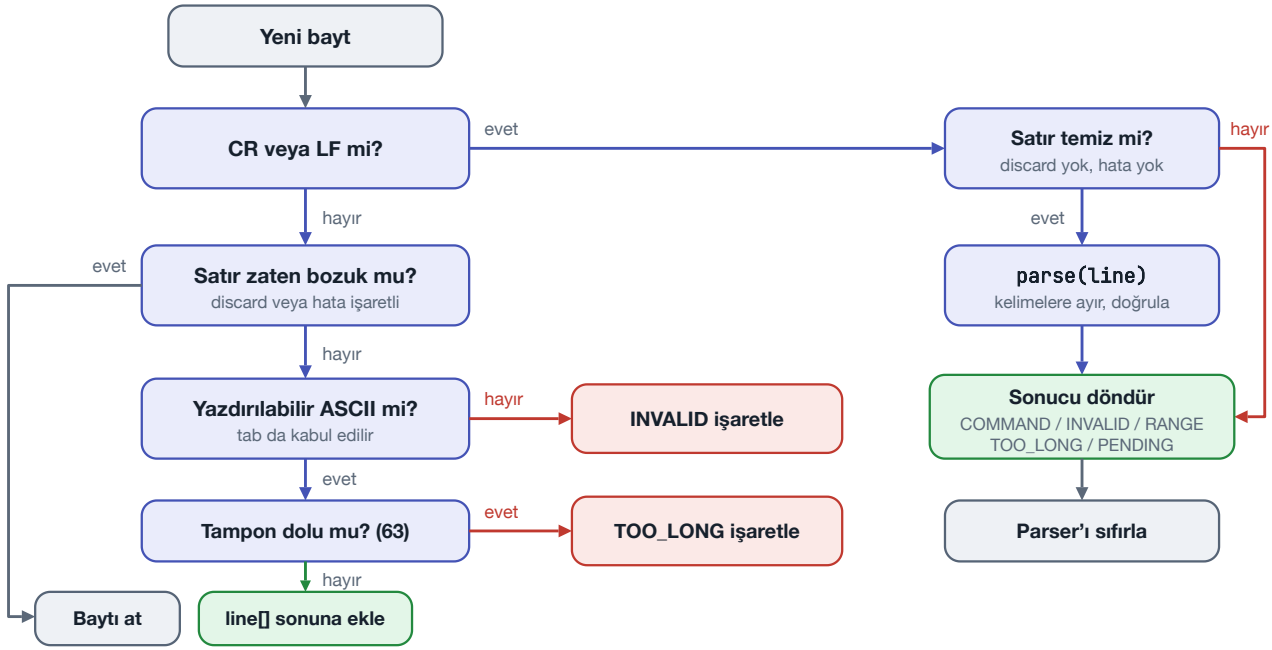
```
355 static void command_poll(void)
356 {
357     if(reply_length) return; /* Backpressure: never execute a command without reply space. */
358     if(rx_loss_notice) {
359         reply_boot = false;
360         const char *error = "ERR rx_data_lost; send a newline, then retry\r\n";
361         strcpy(command_reply, error); reply_length = strlen(error); rx_loss_notice = false;
362         return;
363     }
364     for(unsigned budget = 0; budget < RX_DMA_SIZE && rx_tail != rx_head; ++budget) {
365         if(rx_error_pending || rx_overflow_pending) return;
366         __DMB();
367         uint8_t byte = rx_ring[rx_tail];
368         __DMB();
369         rx_tail = (rx_tail + 1U) % RX_RING_SIZE;
370         watch_led_command_t command = {0};
371         watch_led_input_result_t result = watch_led_commands_push(&command_parser, byte, &command);
372         if(result == WATCH_LED_INPUT_PENDING) continue;
373         if(result == WATCH_LED_INPUT_COMMAND) execute_command(&command);
374         else {
375             reply_boot = false;
376             const char *error = result == WATCH_LED_INPUT_RANGE ? "ERR interval_range 10..60000 step
    10\r\n" :
377                 result == WATCH_LED_INPUT_TOO_LONG ? "ERR line_too_long max=63\r\n" : "ERR
    invalid_command; type HELP\r\n";
378             strcpy(command_reply, error); reply_length = strlen(error);
379         }
380         return;
381     }
382 }
```

> **Komutlar arka arkaya gelirse?** Aynı DMA bildiriminde birden fazla komut olabilir. Hepsi sırayla çalışır ve cevapları aynı sırayla döner. Otomatik bir program yine de her `OK` veya `ERR` cevabını beklemeli; cevap beklemeden sınırsız komut göndermek halkayı doldurabilir.

6 · KOMUT ÇÖZME

Parser: baytlardan komuta

`watch_led_commands_push()` tek seferde tek bayt alır ve bellek olarak yalnızca kendi yapısını kullanır. Bu yüzden veri hangi kaynaktan, kaç parçada gelirse gelsin aynı sonucu üretir.



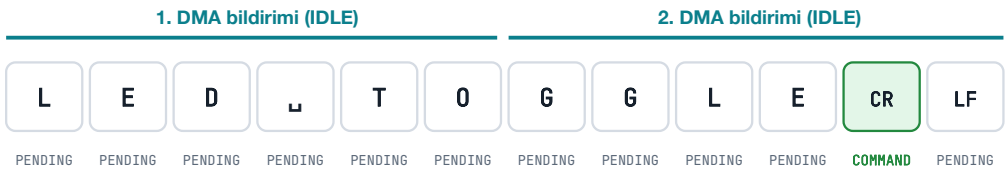
Bir satırda hata çıkarsa satırın geri kalanı satır sonuna kadar atılır ve tek bir hata cevabı üretilir.

App/services/watch_led_commands.c satır 67–85

```
67 watch_led_input_result_t watch_led_commands_push(watch_led_commands_t *parser,
68                                                  uint8_t byte,
69                                                  watch_led_command_t *command)
70 {
71     if(byte == '\r' || byte == '\n') {
72         watch_led_input_result_t result = parser->error;
73         if(!parser->discard && result == WATCH_LED_INPUT_PENDING) {
74             parser->line[parser->length] = '\0';
75             result = parse(parser->line, command);
76         }
77         watch_led_commands_reset(parser, false);
78         return result;
79     }
80     if(parser->discard || parser->error != WATCH_LED_INPUT_PENDING) return WATCH_LED_INPUT_PENDING;
81     if((byte < 32U && byte != '\t') || byte > 126U) parser->error = WATCH_LED_INPUT_INVALID;
82     else if(parser->length == WATCH_LED_COMMAND_MAX) parser->error = WATCH_LED_INPUT_TOO_LONG;
83     else parser->line[parser->length++] = (char)byte;
84     return WATCH_LED_INPUT_PENDING;
85 }
```

Device Studio aynı yolu kullanır. `REQ <seq> HELLO`, `STATUS` ve `BOOT` istekleri de bu parser'dan geçer ve aynı TX DMA ile JSON cevap alır. Tek port olduğu için PuTTY ile Device Studio aynı anda bağlanamaz.

Parçalı veri ve satır doğrulama



IDLE komutun sonu değildir: komutu yalnızca satır sonu tamamlar. Ardından gelen LF boş satır sayılır.

PuTTY satır düzenlemesi kapalıysa her tuş ayrı IDLE üretebilir. Sonuç değişmez; birim testi bir komutu olası her noktadan bölerek bunu doğrular.

Satır sonu gelince `parse()` satırı boşluk ve tab'lardan kelimelere böler, sonra kelime sayısına ve metnine göre komutu seçer. Beklenmedik her şey reddedilir: küçük harf, fazladan parametre, işaretli sayı ya da üstel sayı. İki yeni komut bu zincire eklendi.

App/services/watch_led_commands.c satır 40–46 *yeni satırlar vurgulu*

```
40      else if(count == 2U && !strcmp(words[0], "LED")) {
41          if(!strcmp(words[1], "ON")) command->kind = WATCH_LED_CMD_ON;
42          else if(!strcmp(words[1], "OFF")) command->kind = WATCH_LED_CMD_OFF;
43          else if(!strcmp(words[1], "BLINK")) command->kind = WATCH_LED_CMD_BLINK;
44          else if(!strcmp(words[1], "TOGGLE")) command->kind = WATCH_LED_CMD_TOGGLE;
45          else if(!strcmp(words[1], "STATUS")) command->kind = WATCH_LED_CMD_STATUS;
46          else return WATCH_LED_INPUT_INVALID;
```

App/services/watch_led_commands.c satır 51–62 *aralık doğrulaması ve taşma koruması*

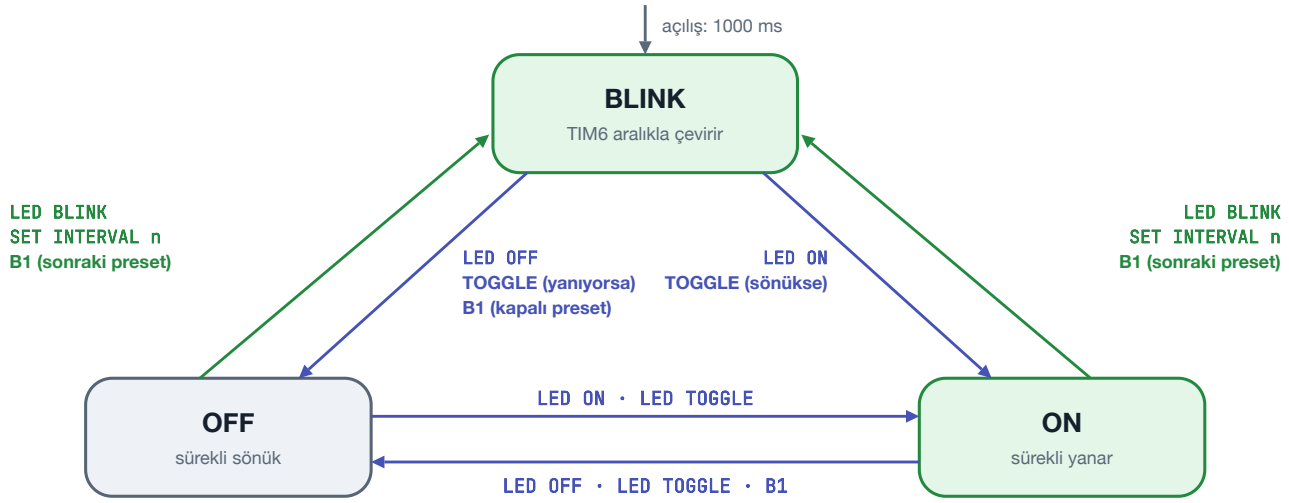
```
51      else if(count == 3U && !strcmp(words[0], "SET") && !strcmp(words[1], "INTERVAL")) {
52          uint32_t value = 0;
53          bool overflow = false;
54          for(const char *p = words[2]; *p; ++p) {
55              if(*p < '0' || *p > '9') return WATCH_LED_INPUT_INVALID;
56              if(value > (UINT32_MAX - (uint32_t)(*p - '0')) / 10U) overflow = true;
57              if(!overflow) value = value * 10U + (uint32_t)(*p - '0');
58          }
59          if(overflow || value < WATCH_LED_INTERVAL_MIN_MS || value > WATCH_LED_INTERVAL_MAX_MS || value %
    10U)
60              return WATCH_LED_INPUT_RANGE;
61          command->kind = WATCH_LED_CMD_INTERVAL;
62          command->interval_ms = value;
```

Girdi	Sonuç	Neden
`LED ON`	**COMMAND**	Geçerli komut
`  HELP`	**COMMAND**	Baştaki ve sondaki boşluklar önemsizdir
`led on`	**INVALID**	Komutlar büyük harflidir
`LED TOGGLE extra`	**INVALID**	Fazladan parametre kabul edilmez
`SET INTERVAL 15`	**RANGE**	10'un katı değil
`SET INTERVAL 4294967296`	**RANGE**	32 bit taşması yakalanır
`SET INTERVAL -10`	**INVALID**	Yalnızca rakam kabul edilir
64 karakterden uzun satır	**TOO_LONG**	Sınır 63 karakterdir; sondaki parça ayrı komut gibi çalışmaz

7 · LED

LED modları ve komutun uygulanması

LED'in davranışı tek bir mod değişkeninde tutulur. Komutlar ve B1 butonu yalnızca bu modu değiştirir, TIM6 kesmesi de mod `BLINK` ise LED'i çevirir.



`LED TOGGLE` LED'in o anki görünen durumunu tersine çevirir ve yanıp sönmeyi durdurur. B1 ise kendi preset dizisinde ilerler.

Core/Src/led_button_demo.c satır 325–346 *execute_command, yeni satırlar vurgulu*

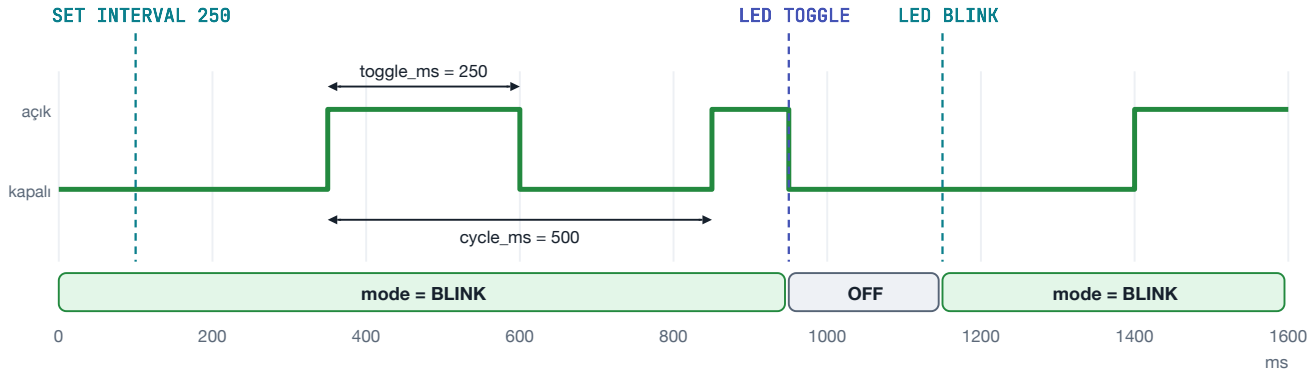
```
325     uint32_t primask = __get_PRIMASK();
326     __disable_irq();
327     switch(command->kind) {
328     case WATCH_LED_CMD_ON:
329         led_mode = LED_MODE_ON; led_on = 1; BSP_LED_On(LED_GREEN); break;
330     case WATCH_LED_CMD_OFF:
331         led_mode = LED_MODE_OFF; led_on = 0; BSP_LED_Off(LED_GREEN); break;
332     case WATCH_LED_CMD_TOGGLE:
333         /* Invert what the LED shows now; a running blink stops on the new state. */
334         if(led_on) { led_mode = LED_MODE_OFF; led_on = 0; BSP_LED_Off(LED_GREEN); }
335         else { led_mode = LED_MODE_ON; led_on = 1; BSP_LED_On(LED_GREEN); }
336         break;
337     case WATCH_LED_CMD_INTERVAL:
338         blink_interval_ms = command->interval_ms;
339         /* SET INTERVAL also starts blinking, with a full first interval. */
340         /* fall through */
341     case WATCH_LED_CMD_BLINK:
342         led_mode = LED_MODE_BLINK; blink_elapsed_ms = 0; break;
343     case WATCH_LED_CMD_RESET_PRESSES: button_presses = 0; break;
344     default: break;
345     }
346     __set_PRIMASK(primask);
```

Neden kesmeler kısa süre kapatılıyor? Mod, `led_on` ve GPIO birlikte değişmeli. TIM6 tam ortada çalışıp LED'i çevirseydi `led_on` ile gerçek pin durumu ayrışabilirdi. `__disable_irq()` bu birkaç satırı bölünmez yapar, cevap metni ise kesmeler açıldıktan sonra hazırlanır.

8 · ZAMANLAMA

TIM6: yanıp sönme ve buton

TIM6 her **10 ms**'de bir kesme üretir. Aralıkların 10'un katı olmasının nedeni de budur. Yanıp sönme kenarları ana döngüye bağlı değildir, yani ekran çizimi LED zamanlamasını bozmaz.



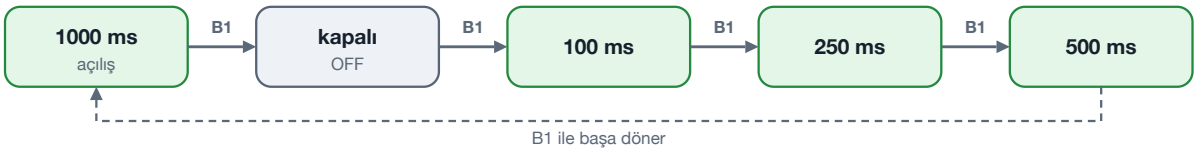
`SET INTERVAL 250`: her 250 ms'de bir durum değişir, tam çevrim 500 ms'dir. Yanıp sönme başlarken ilk geçiş tam bir aralık sonra olur.

Core/Src/led_button_demo.c satır 233–242

```
233     /* Blink edges remain timer-driven; ON/OFF commands apply atomically below. */
234     uint32_t interval = blink_interval_ms;
235     if(led_mode == LED_MODE_BLINK) {
236         if(!speed_changed) blink_elapsed_ms += TIMER_STEP_MS;
237         if(blink_elapsed_ms >= interval) {
238             blink_elapsed_ms = 0;
239             BSP_LED_Toggle(LED_GREEN);
240             led_on ^= 1U;
241         }
242     }
```

B1 butonu

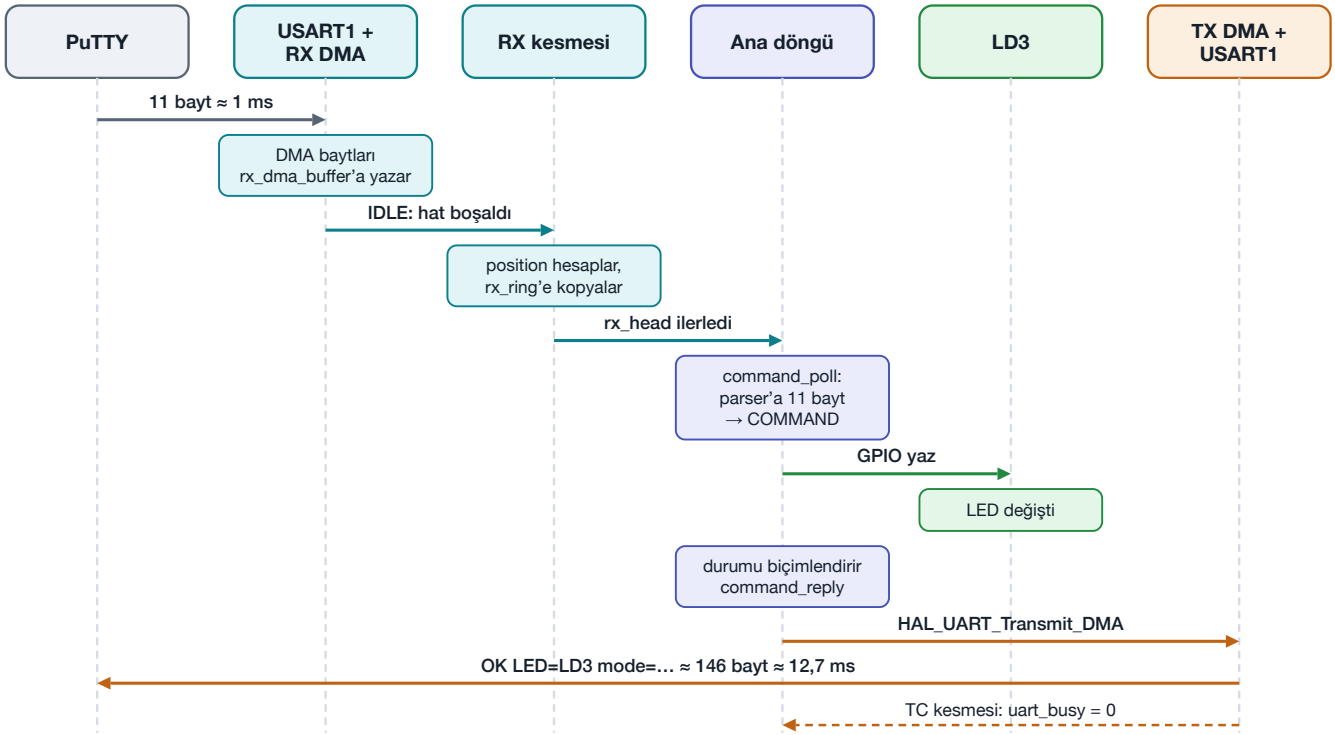
Buton kenarı **EXTI13** kesmesini tetikler ama hemen işlenmez. TIM6, seviyenin 40 ms boyunca sabit kaldığını görünce basışı kabul eder. Böylece buton sıçraması tek basışı birkaç basış gibi saydırmaz. Seri komutlar B1'in preset sırasını değiştirmez.



Bir sonraki B1 basışı, araya hangi seri komut girerse girsin dizideki sıradaki preset'i uygular.

9 · UÇTAN UCA

Komuttan cevaba zaman çizgisi



`LED TOGGLE` ve Enter için. LED, cevap gönderilmeden önce değişir. Süreler 115200 baud'dan hesaplanmıştır; dikey aralıklar ölçekli değildir.

Cevap gönderimi

TX DMA çalışırken `uart_message` tamponuna dokunulmaz. Bu sırada oluşan cevap `command_reply`'de bekler. Cevaplar, her saniye gelen durum satırından önceliklidir.

Core/Src/led_button_demo.c satır 424–446 — *led_button_demo_poll, gönderim bölümü*

```
424     if(uart_busy || (!reply_length && !status_pending)) return;
425     int length;
426     if(reply_length) {
427         length = (int)reply_length;
428         memcpy(uart_message, command_reply, reply_length);
429         reply_length = 0;
430         uart_boot = reply_boot;
431         reply_boot = false;
432     } else {
433         uart_boot = false;
434         status_pending = 0;
435         length = format_status(uart_message, sizeof uart_message, "");
436     }
437     if(length <= 0 || (size_t)length >= sizeof(uart_message)) {
438         ++uart_errors;
439         return;
440     }
441     uart_started_at = HAL_GetTick();
442     uart_message_bytes=(uint32_t)length;
443     uart_busy = 1;
444     __DMB();
445     if(HAL_UART_Transmit_DMA(&led_demo_uart, (const uint8_t *)uart_message, (uint16_t)length) != HAL_OK)
446         uart_error_pending = 1;
```

10 · DAYANIKLILIK

Hatalar ve toparlanma

Terminalde görülen	Anlamı	Ne yapılır
`ERR invalid_command; type HELP`	Bilinmeyen komut, küçük harf, kontrol karakteri	Komutu büyük harfle yeniden yaz
`ERR interval_range 10..60000 step 10`	Aralık sınır dışında veya 10'un katı değil	Geçerli bir değer gönder
`ERR line_too_long max=63`	Satır 63 karakteri aştı	Satırı kısalt
`ERR rx_data_lost; send a newline, then retry`	Halka taştı veya UART/DMA hatası oluştu	Önce boş Enter, sonra komut

Hatalı bir komut LED ayarını değiştirmez. `rx_data_lost` durumunda parser satır sonuna kadar her şeyi atar; böylece yarım kalan bir komutun son parçası yeni bir komut gibi çalışamaz. Sayaçlar durum satırında görünür: `uart_errors`, `rx_errors`, `rx_overflows`.

Kim, nerede çalışır?

Bağlam	Öncelik	Görev
TIM6	6	Buton debounce, yanıp sönme kenarları, 1 saniyelik durum bayrağı
EXTI13 (B1)	6	Buton kenarının zamanını kaydeder
GPDMA1 Ch1 + USART1	7	HT, TC ve IDLE ile baytları halkaya kopyalar; TC ile TX'i bitirir
GPDMA1 Ch0	7	TX DMA bitince USART TC'yi devreye alır
Ana döngü	Kesmesiz	Parser, komut uygulama, cevap biçimlendirme, LVGL ekran çizimi

Cortex-M'de küçük sayı yüksek önceliktir. RX DMA ve USART aynı öncelikte olduğu için iki üretici callback iç içe çalışamaz.

Sorun giderme

Belirti	Çözüm
`/dev/cu.usbmodem*` bulunamıyor	Kablonun CN5 (ST-LINK) portunda ve veri taşıyan bir kablo olduğunu kontrol et
Port açılamıyor	Device Studio veya aynı portu kullanan başka terminali kapat
Yazdıklarım görünmüyor	PuTTY'yi `tools/led-terminal.sh` ile aç veya Local echo'yu Force on yap
Backspace sonrası `ERR invalid_command`	Local line editing'i Force on yap
Durum satırları yazdığımın arasına giriyor	Normaldir: satır düzenlemesi açıkken komut Enter'da bozulmadan gider
`LED TOGGLE` → `ERR invalid_command`	Kartta eski firmware vardır; güncel `.dfw` paketini yükle

11 · DOĞRULAMA

Neler test edildi?

69 / 69
Parser birim testi
Bilgisayarda derlenir, UndefinedBehaviorSanitizer açıktır. Bir komutu olası her noktadan böler, taşma ve bozuk girdileri dener.

88 / 88
ARM + HAL simülasyonu
Gerçek derlenmiş firmware Unicorn'da çalışır. Kesmeler ve HAL gerçek koddur; DMA ve periferik register'lar modellenir.

Yapılmadı
Fiziksel kart testi
`LED TOGGLE` ve `LED STATUS` henüz gerçek kartta denenmedi. Bu derleme sırasında kart bilgisayara bağlı değildi.

```
python3 output/verify-led-commands.py
python3 output/build-macos.py
python3 tools/package-firmware.py
python3 output/verify-led-button.py
```

Kartta ilk deneme

1. `HELP` yaz: cevapta `TOGGLE` görünüyorsa yeni firmware yüklenmiştir.
2. `LED TOGGLE` yaz: LD3 durum değiştirmeli ve yanıp sönmeyi bırakmalı. Tekrar yaz: yine değişmeli.
3. `LED STATUS` yaz: `mode` ve `state` LED'in gördüğün durumuyla aynı olmalı.
4. `SET INTERVAL 250` yaz: LED saniyede iki kez yanıp sönmeli (tam çevrim 500 ms).

Dosya haritası

Dosya	İçerik
`App/services/watch_led_commands.c/.h`	Donanımdan bağımsız satır çerçeveleme ve komut parser'ı
`Core/Src/led_button_demo.c`	UART ve DMA kurulumu, kesme callback'leri, komut uygulama, TIM6, B1
`Core/Src/device_runtime.c`	Device Studio için JSON cevaplar ve BOOT isteği
`Core/Src/main.c`	`led_button_demo_poll()` çağıran ana döngü
`docs/led-uart-commands.md`	Protokolün ayrıntılı başvuru metni
`output/test-led-commands.c`	Parser birim testi
`output/verify-led-button.py`	ARM + HAL simülasyon testi